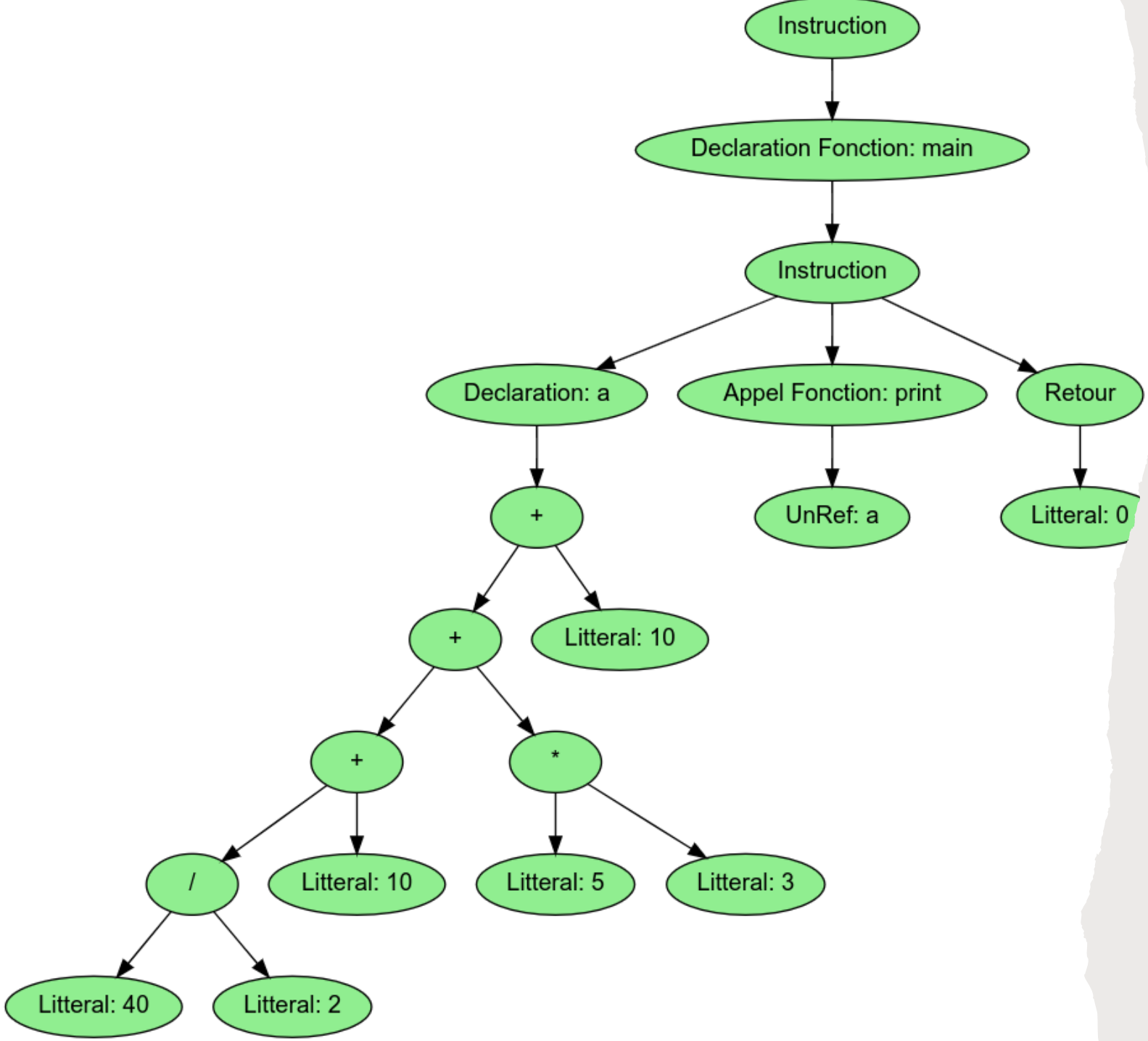

Vue d'ensemble sur le projet

Sprint 1 (Terminé)

- Lire des fichiers sources Prysma et les traiter comme du texte Lire des fichiers sources Prysma et les traiter comme du texte Lire des fichiers sources Prysma et les traiter comme du texte
- Supporter les types entiers (int), flottants (float), booléens (bool)
- Compiler les opérations arithmétiques de base (+, -, *, /) vers du code LLVM
- Permettre au développeur de déclarer des variables et les utiliser dans des expressions
- Valider la syntaxe du code et rejeter les structures invalides via la construction de l'AST
- Supporter les fonctions récursives et un type de retour.
- Signaler les erreurs de syntaxe précisément (numéro de ligne, type d'erreur) plutôt que planter silencieusement
- Ajouter la coloration syntaxique du langage Prysma dans Visual Studio Code

Sprint 2 (Terminé - Focus actuel)

- Exporter l'AST en format texte et graphique (Graphviz) pour faciliter le débogage
- Refonte complète de la gestion mémoire pour une performance maximale
- supporter les opérantes && || == != %
- Supporter l'instruction if,
- Supporter l'instruction while, passer par la pile avec alloca
- Supporter les tableaux programmés en LLVM IR
- Supporter le type string
- Supporter les includes, en utilisant les techniques du langage de programmation C #include "racine/fichier.h"
- Mettre en place un système de compilation multi-thread performant

Sprint 3 (À venir)

- Supporter l'approche orientée objet avec des classes, polymorphisme fait avec une vtable.
- Le Layout de Données (La Structure)
- Les Contrôles d'Accès (Visibilité)
- l'Encapsulation et le Pointeur this
- Le Cycle de Vie (Constructeurs & Destructeurs)
- L' Héritage (La Hiérarchie de Classes)
- Le Polymorphisme (La VTable)
- Analyse syntaxique abstraite pour la logique du langage Prysma, ce qui évitera les problèmes de logique sémantique exemple : declare int a = "string";
- Écrire une bibliothèque de structures de données (listes chaînées, arbres binaires, piles, files)

Choix technologiques effectués

- Gestion de la mémoire (LLVM BumpPtrAllocator)
- Système variadique C pour l'affichage
- Langage de programmation C++
- LLVM

Ce que j'ai choisi de développer durant l’itération

Tableaux et String

Logique de contrôle : La boucle while et les conditions if/else

Multi-threading et Includes

Tests automatisés robustes

Mes bons coups

- Le gain de performance extrême
- Implémentation stable des fonctionnalités
- Test solide

Défis rencontrés

- Le cauchemar du Multi-thread avec LLVM
- La récursivité infinie des appels
- La difficulté de LLVM

Ce qui reste à développer pour la prochaine itération

Programmation Orientée Objet.

Implémenter le système de classes et le polymorphisme en utilisant une "vtable" (table de méthodes virtuelles) générée en LLVM IR.

Écrire la bibliothèque standard de Prysma (Listes chaînées, Arbres binaires) pour prouver que mon compilateur est capable de compiler des structures de données complexes par lui-même.